

1. **Title:** GitHub Task: Lexical Analyzer & Token Counter
2. **Objective:** Develop a program that reads a source-code file and performs lexical analysis by identifying and counting different types of tokens.
3. **Task:** Students should implement a program in Python/C/C++/Java that identifies:

- Keywords
- Identifiers
- Operators
- Constants/Literals
- Separators/Delimiters
- Special symbols
- Comments

4. Algorithm

1. Start the program.
2. Read the source-code filename from the command line.
3. Open the source-code file.
4. Read the complete source code.
5. Scan the source code using regular-expression patterns.
6. Identify single-line and multi-line comments.
7. Identify string and character literals.
8. Identify multi-character operators such as ==, !=, <=, >=, ++, and --.
9. Identify keywords.
10. Identify identifiers.
11. Identify numeric constants.
12. Identify operators.
13. Identify separators and delimiters.
14. Identify special symbols.
15. Store every recognized token along with its token type.
16. Count the number of tokens in every category.
17. Display the token and its classification.
18. Display the total token count.
19. Stop the program.

5. Source Code

The following Python program implements the lexical analyzer.

```
import re
import sys
```

```
KEYWORDS = {
    "auto", "break", "case", "char", "const", "continue",
    "default", "do", "double", "else", "enum", "extern",
    "float", "for", "goto", "if", "int", "long",
    "register", "return", "short", "signed", "sizeof",
    "static", "struct", "switch", "typedef", "union",
```

}

}

}

}

```
def classify_token(token):
```

```
if token in KEYWORDS:
    return "Keyword"
```

```
if token in OPERATORS:
    return "Operator"
```

```
if token in SEPARATORS:
    return "Separator"
```

```
if len(token) >= 2 and token[0] == "'" and token[-1] == "':
    return "String Literal"
```

```
if len(token) >= 3 and token[0] == "" and token[-1] == "":
    return "Constant"
```

```
number_pattern = (
    r"([0-9a-fA-F]+|"
    r"\d+(\.\d*)?([eE][+-]?\d+)?|"
    r"\.\d+([eE][+-]?\d+)?)")
```

```
if re.fullmatch(number_pattern, token):
```

```
    return "Constant"
```

```
if re.fullmatch(r"[A-Za-z_][A-Za-z0-9_]*", token):  
    return "Identifier"
```

```
if token in SPECIAL_SYMBOLS:  
    return "Special Symbol"
```

```
return "Special Symbol"
```

```
def tokenize(source_code):
```

```
    pattern = re.compile(  
        r"""  
        //.*?$                |  
        \*[ \s\S]*?\*/        |  
  
        "(?:\\.|[^\"])*"      |  
        '(?:\\.|[^\'])*'      |  
  
        ==|!=|<=|>=|\+|\+|--|\+=|-=|\*=|/=|%|=|  
        &&|\||<<|>>          |  
  
        [A-Za-z_][A-Za-z0-9_]*    |  
  
        0[xX][0-9a-fA-F]+        |  
        \d+(\.\d*)?([eE][+-]?[d+])? |  
        \.\d+                    |  
  
        [\+\.*/%=<>!&|^~]        |  
  
        [( )\[\]\{\};,:.]        |  
  
        [#$@?\\]  
        """,  
        re.VERBOSE | re.MULTILINE  
    )
```

```
    tokens = []  
    comments = []
```

```
    for match in pattern.finditer(source_code):
```

```
        token = match.group(0)
```

```
        if token.startswith("//") or token.startswith("/*"):  
            comments.append(token)
```

```
        else:  
            token_type = classify_token(token)
```

```

        tokens.append((token, token_type))

    return tokens, comments

def display_results(tokens, comments):

    print()
    print("TOKEN TYPE")
    print("-" * 50)

    for token, token_type in tokens:
        print(f"{token:<25} {token_type}")

    print()
    print("-" * 50)
    print("Token Count")
    print("-" * 50)

    counts = {
        "Keyword": 0,
        "Identifier": 0,
        "Operator": 0,
        "Constant": 0,
        "String Literal": 0,
        "Separator": 0,
        "Special Symbol": 0
    }

    for token, token_type in tokens:
        if token_type in counts:
            counts[token_type] += 1

    print(f"Keywords       : {counts['Keyword']}")
    print(f"Identifiers    : {counts['Identifier']}")
    print(f"Operators      : {counts['Operator']}")
    print(f"Constants      : {counts['Constant']}")
    print(f"String Literals : {counts['String Literal']}")
    print(f"Separators     : {counts['Separator']}")
    print(f"Special Symbols : {counts['Special Symbol']}")
    print(f"Comments       : {len(comments)}")

def main():

    if len(sys.argv) != 2:
        print("Usage: python lexical_analyzer.py input.txt")
        return

    filename = sys.argv[1]

```

```
try:

    with open(filename, "r", encoding="utf-8") as file:
        source_code = file.read()

    tokens, comments = tokenize(source_code)

    display_results(tokens, comments)

except FileNotFoundError:
    print(f"Error: File '{filename}' not found.")

except Exception as error:
    print("Error:", error)

if __name__ == "__main__":
    main()
```

6. Sample Input

The following C program is used as the sample input:

```
int sum = a + b;
float average = sum / 2.0;

// Calculate average
if (average > 50)
    printf("Pass");
```

7. Sample Input

Enter source file name: input.txt

TOKEN TYPE

```
-----
int           Keyword
sum           Identifier
=             Operator
a             Identifier
+             Operator
b             Identifier
;             Separator
float         Keyword
average       Identifier
=             Operator
sum           Identifier
/             Operator
2.0           Constant
;             Separator
// Calculate average Comment
if            Keyword
(             Separator
average       Identifier
>             Operator
50            Constant
)             Separator
printf        Identifier
(             Separator
"Pass"        String Literal
)             Separator
;             Separator
-----
```

Token Count

```
Keywords      : 3
Identifiers   : 7
Operators     : 5
Constants     : 2
String Literals : 1
Separators    : 7
Special Symbols : 0
Comments      : 1
```

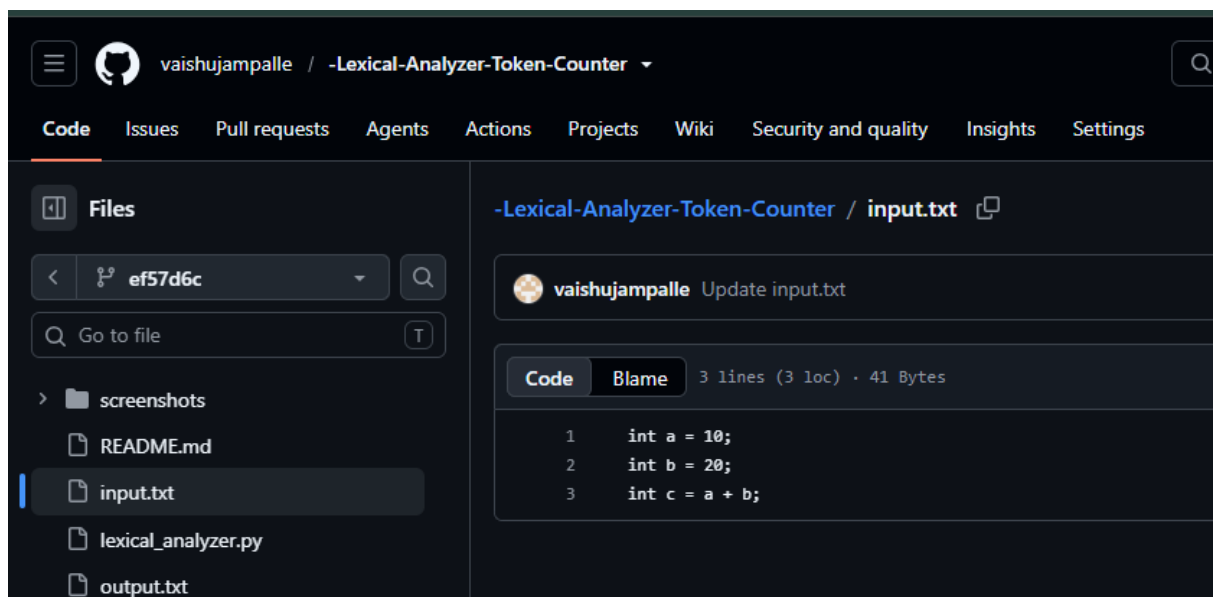
8. Token Classification

Token	Token Type	Explanation
int	Keyword	Reserved word used to declare an integer
float	Keyword	Reserved word used to declare a floating-point variable
if	Keyword	Conditional statement keyword
sum	Identifier	Name of a variable
average	Identifier	Name of a variable
printf	Identifier	Function name
=	Operator	Assignment operator
+	Operator	Addition operator
/	Operator	Division operator
>	Operator	Greater-than operator
2.0	Constant	Floating-point numeric constant
50	Constant	Integer numeric constant
"Pass"	String Literal	Sequence of characters enclosed in double quotes
(	Separator	Opening parenthesis
)	Separator	Closing parenthesis
;	Separator	Statement terminator
//	Comment	Single-line comment

9. Test Cases

Test Case 1: Basic Arithmetic

Input:



The screenshot shows a GitHub repository interface for a project named "-Lexical-Analyzer-Token-Counter" by user "vaishujampalle". The "Files" tab is selected, showing a directory structure with "screenshots", "README.md", "input.txt", "lexical_analyzer.py", and "output.txt". The "input.txt" file is selected, and its content is displayed in the main area. The file was updated by "vaishujampalle". The code content is as follows:

```
1  int a = 10;
2  int b = 20;
3  int c = a + b;
```

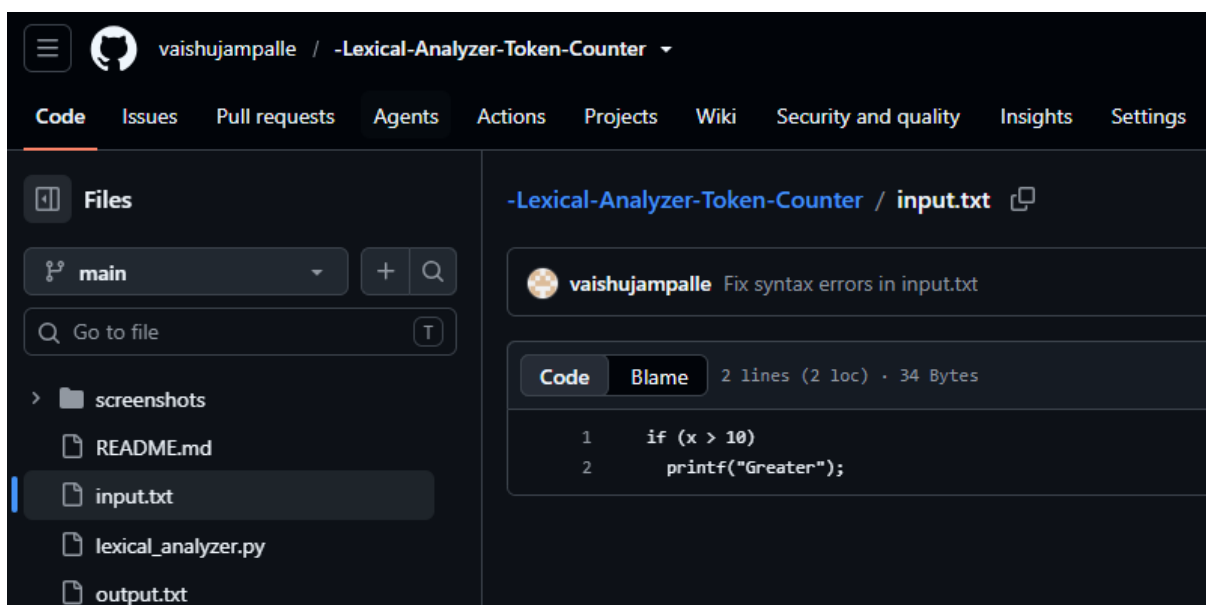
Output:

```
vaishujampalle@ubuntu: ~$ python3 -Lexical-Analyzer-Token-Counter.py
TOKEN TYPE
-----
int      Keyword
a        Identifier
=        Operator
10       Constant
;        Separator
int      Keyword
b        Identifier
=        Operator
20       Constant
;        Separator
int      Keyword
c        Identifier
=        Operator
a        Identifier
+        Operator
b        Identifier
;        Separator
-----

Token Count
Keywords      : 3
Identifiers   : 5
Operators     : 4
Constants     : 2
String Literals : 0
Separators    : 3
Special Symbols : 0
Comments      : 0
```

Test Case 2: Conditional Statement

Input:



The screenshot shows a GitHub repository page for 'vaishujampalle / -Lexical-Analyzer-Token-Counter'. The 'Files' tab is active, showing a list of files: 'screenshots', 'README.md', 'input.txt', 'lexical_analyzer.py', and 'output.txt'. The 'input.txt' file is selected, showing its commit history. The commit by 'vaishujampalle' is highlighted, with the message 'Fix syntax errors in input.txt'. The code for 'input.txt' is displayed, showing two lines of C code: '1 if (x > 10)' and '2 printf("Greater");'.

```
1  if (x > 10)
2  printf("Greater");
```


Output:

```
TOKEN TYPE
-----
if          Keyword
(           Separator
x           Identifier
>           Operator
10          Constant
)           Separator
printf      Identifier
(           Separator
"Greater"   String Literal
)           Separator
;           Separator
-----

Token Count
Keywords      : 1
Identifiers   : 2
Operators     : 1
Constants     : 1
String Literals : 1
Separators    : 5
Special Symbols : 0
Comments      : 0
```

10.Conclusion

The Lexical Analyzer and Token Counter project successfully demonstrates the basic process of lexical analysis used in compiler design.

The Python program reads source code from an input file and identifies different types of tokens, including keywords, identifiers, operators, constants, string literals, separators, special symbols, and comments.

The program also counts the occurrence of each token category and displays the results in an organized format. This project provides a basic understanding of how a compiler's lexical analysis phase processes source code and converts it into meaningful tokens.